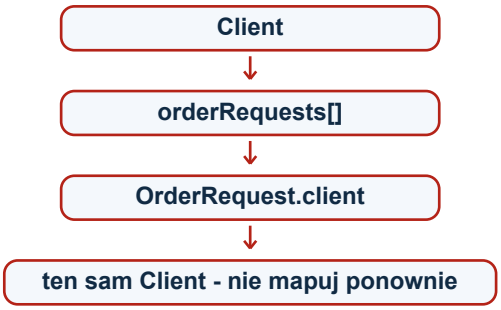
MapStruct bez pętli

Przewodnik po mapowaniu Entity ↔ Domain ↔ DTO na przykładach Client, OrderRequest, Order i DeliveryAddress z projektu ZP Tank.

> Cel
> Po lekturze będziesz umiał zaprojektować DTO, rozpoznać relację cykliczną, świadomie użyć uses, @Named, qualifiedByName, wariantów mapperów oraz zdecydować, kiedy @Context ma sens.

Najważniejsza zasada

Encje i obiekty domain mogą zawierać relacje dwukierunkowe. Pojedynczy wynik mappera nie powinien jednak rozwijać pełnej relacji w obie strony.



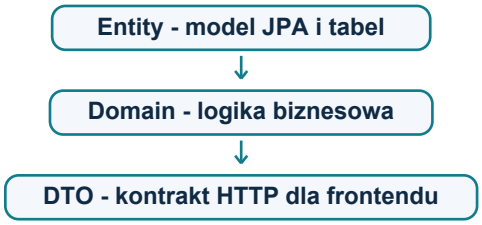
> Zapamiętaj
> Nie twórz odpowiedzi ani mapowania: pełne A → pełne B → ponownie pełne A. Zamiast powrotu użyj wersji Summary, ID albo świadomie pomiń pole.

Spis treści

1. Trzy warstwy modelu 2. Skąd bierze się StackOverflowError 3. DTO Summary/Details/Command 4. API mappery i uses 5. Entity ↔ Domain 6. Endpointy klienta 7. Zapis relacji po ID 8. @Context 9. Plan wdrożenia

1. Trzy warstwy, trzy odpowiedzialności

W projekcie te same dane przechodzą przez trzy różne reprezentacje. Nie należy oczekiwać, że będą kopiami 1:1.

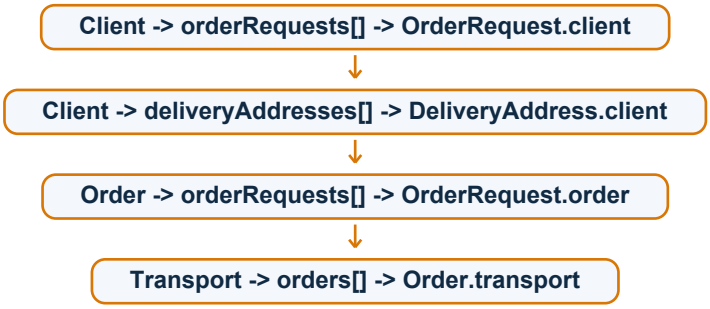


Warstwa	Przykład	Za co odpowiada
Entity	ClientEntity, OrderRequestEntity	Mapowanie tabel, klucze obce, @ManyToOne, @OneToMany, lazy loading.
Domain	Client, OrderRequest	Model używany przez serwisy i reguły biznesowe.
DTO	ClientSummaryDTO, CreateOrderRequestDTO	Dokładny format wejścia lub odpowiedzi HTTP.

> Wniosek
> To, że Client ma listę zleceń, nie oznacza, że każdy ClientDTO musi ją mieć. DTO projektujesz pod konkretny ekran i endpoint.

Relacje obecne w Twoim modelu

W Twoich klasach występują m.in. Client ↔ OrderRequest, Client ↔ DeliveryAddress, Order ↔ OrderRequest oraz Transport ↔ Order. Każda z nich może utworzyć rekurencję, jeśli MapStruct mapuje pola automatycznie w obu kierunkach.



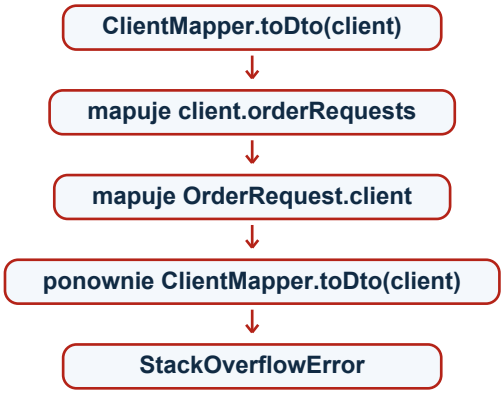
2. Dlaczego pojawia się StackOverflowError

MapStruct nie pobiera sam danych z bazy. Bierze otrzymany obiekt źródłowy i tworzy obiekt docelowy, wywołując mapowania dla kolejnych pól. Jeśli graf jest zamkniętym kołem, wywołania metod nigdy się nie kończą.

Przykład niebezpiecznego DTO

```
class ClientDTO {
List<OrderRequestDTO> orderRequests;
}

class OrderRequestDTO {
ClientDTO client;
}
```



> Istotne
> Tak samo może się stać na poziomie Entity ↔ Domain. Błąd nie jest specyficzny dla DTO. Powstaje w każdym mapperze, który przechodzi po pełnym cyklu.

Automatyczne mapowanie nie jest świadomym wyborem

Gdy pola mają zgodne nazwy i typy, MapStruct często wygeneruje pomocnicze metody automatycznie. Jest to świetne dla klas płaskich, np. Product → ProductDTO. Przy relacjach zagnieżdżonych nie kontrolujesz wtedy, czy wygenerowana metoda nie rozwinie za dużo pól.

```
// Bez ■wiadomie dobranego wariantu MapStruct mo■e przej■■
// z ClientDTO do OrderRequestDTO, potem znów do ClientDTO.
ClientDTO toDto(Client client);
```

3. Projekt DTO: Summary, Details i Command

Najlepszym rozwiązaniem jest tworzenie DTO o różnej zawartości. Nazwa powinna mówić, dla jakiego rodzaju odpowiedzi klasa istnieje.

A. DTO skrócone (Summary)

```
public class ClientSummaryDTO {
private Long id;
private String name;
private String taxId;
}

public class OrderRequestSummaryDTO {
private Long id;
private OrderRequestStatus status;
private ProductDTO product;
private BigDecimal quantity;
private LocalDate executionDate;
}
```

Summary nie zawiera kolekcji odwrotnych. Dzięki temu można bezpiecznie osadzić klienta w zleceniu albo zlecenie w kliencie.

B. DTO szczegółów (Details)

```
public class OrderRequestDetailsDTO {
private Long id;
private OrderRequestStatus status;
private LocalDateTime createdAt;
private ClientSummaryDTO client;
private ProductDTO product;
private BigDecimal quantity;
private BigDecimal price;
private LocalDate executionDate;
private String notes;
private OrderSummaryDTO order;
}
```

C. DTO żądania (Command)

```
public class CreateOrderRequestDTO {
private Long clientId;
private Long productId;
private BigDecimal quantity;
private BigDecimal price;
private LocalDate executionDate;
private String notes;
}
```

> Dobra praktyka
> Dla POST/PATCH przesyłaj ID relacji, np. clientId - nie ClientDTO. Dla GET zwróć ClientSummaryDTO, jeśli frontend ma wyświetlić nazwę klienta.

4. API mappery - jak je pisać

Dla jednej klasy domain możesz mieć wiele metod mapujących do różnych DTO. To jest oczekiwany styl MapStruct.

```
import org.mapstruct.Mapper;
import org.mapstruct.Named;

@Mapper(componentModel = "spring")
public interface ClientMapper {

@Named("clientSummary")
ClientSummaryDTO toSummaryDto(Client client);

ClientDetailsDTO toDetailsDto(Client client);
}
```

Metoda toSummaryDto przepisze tylko pola obecne w ClientSummaryDTO. Ponieważ Summary nie ma orderRequests, MapStruct nie wejdzie w listę zleceń.

uses - dostęp do innych mapperów

```
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

@Mapper(
componentModel = "spring",
uses = {ClientMapper.class, ProductMapper.class, OrderMapper.class}
)
public interface OrderRequestMapper {

@Mapping(target = "client", qualifiedByName = "clientSummary")
@Mapping(target = "order", qualifiedByName = "orderSummary")
OrderRequestDetailsDTO toDetailsDto(OrderRequest orderRequest);
}
```

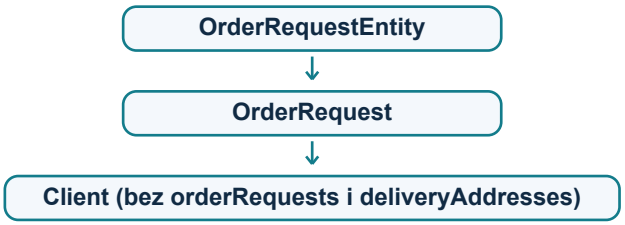
uses oznacza: "MapStruct może skorzystać z metod tych mapperów." qualifiedByName oznacza: "Wybierz dokładnie metodę oznaczoną tą nazwą."

> Nie myl pojęć
> Samo uses nie usuwa pętli. Ono tylko daje dostęp do mappera. Pętlę przerywa wybór DTO Summary, @Mapping(ignore = true) albo metody wariantowej bez relacji zwrotnej.

5. Entity ↔ Domain - ten sam problem, inny cel

Encje i domain mogą zawierać pełne relacje, ale mapowanie ich w całości nadal może wejść w pętlę. Używaj wariantów metod zależnych od tego, co pobierasz.

A. Pobierasz pojedyncze zlecenie wraz z klientem



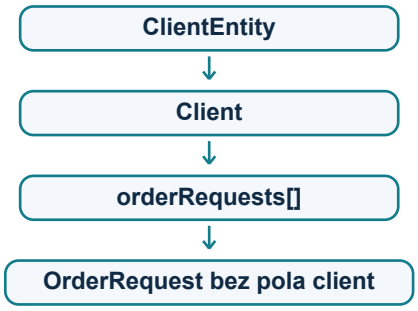

```
@Mapper(componentModel = "spring")
public interface ClientEntityMapper {

@Named("clientWithoutRelations")
@Mapping(target = "orderRequests", ignore = true)
@Mapping(target = "deliveryAddresses", ignore = true)
Client toDomainWithoutRelations(ClientEntity entity);
}
```

```
@Mapper(componentModel = "spring", uses = ClientEntityMapper.class)
public interface OrderRequestEntityMapper {

@Mapping(target = "client", qualifiedByName = "clientWithoutRelations")
OrderRequest toDomain(OrderRequestEntity entity);
}
```

B. Pobierasz klienta z jego zleceniami




```
@Named("orderRequestWithoutClient")
@Mapping(target = "client", ignore = true)
@Mapping(target = "order", ignore = true)
OrderRequest toDomainWithoutClient(OrderRequestEntity entity);
```

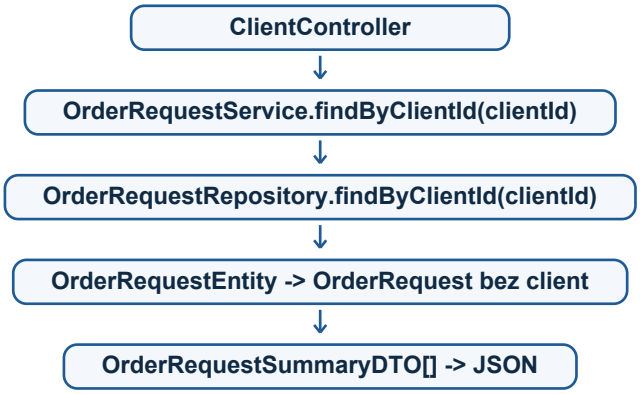
> Domyślne mapowanie
> Warto mieć wariant płytki jako najczęściej używany. Warianty z kolekcjami twórz tylko wtedy, gdy konkretny przypadek użycia naprawdę ich potrzebuje.

6. Jak wyświetlić wszystkie zlecenia klienta

Przerwanie cyklu nie blokuje pobierania listy zleceń. Nie trzeba najpierw pobierać pełnego klienta i przechodzić po jego grafie. Zwykle lepiej wykonać bezpośrednie zapytanie po clientId.

Endpoint

```
GET /clients/{clientId}/order-requests
```




```
public interface OrderRequestJpaRepository
extends JpaRepository<OrderRequestEntity, Long> {

List<OrderRequestEntity> findByClientId(Long clientId);
}
```

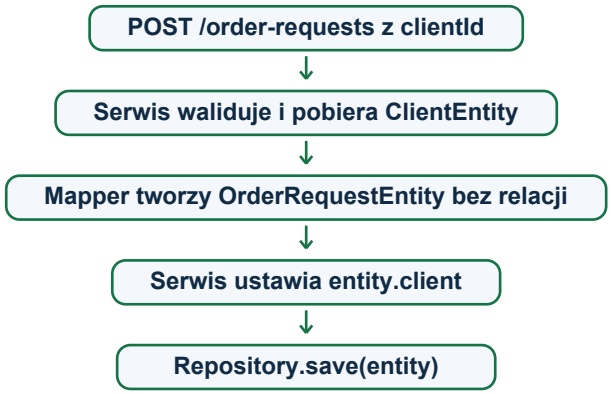
Frontend podał już clientId w URL, dlatego każde zlecenie na liście nie potrzebuje ponownie pełnego klienta.

```
[
{"id": 101, "status": "PENDING", "quantity": 24000},
{"id": 102, "status": "ACCEPTED", "quantity": 12000}
]
```

> Alternatywa
> GET /clients/{id} może zwracać ClientDetailsDTO z List<OrderRequestSummaryDTO>. To nadal jest bezpieczne, ponieważ OrderRequestSummaryDTO nie zawiera ClientDetailsDTO.

7. Zapis relacji - ID w DTO, encja w serwisie

MapStruct nie powinien wyszukiwać ClientEntity w repozytorium na podstawie clientId. Mapowanie danych i pobieranie danych z bazy to dwie różne odpowiedzialności.




```
ClientEntity client = clientJpaRepository.getReferenceById(request.getClientId());
ProductEntity product = productJpaRepository.getReferenceById(request.getProductId());

OrderRequestEntity entity = orderRequestEntityMapper
.toEntityWithoutRelations(domain);

entity.setClient(client);
entity.setProduct(product);
orderRequestJpaRepository.save(entity);
```

Jeżeli potrzebujesz własnego komunikatu "klient nie istnieje", użyj findById(...).orElseThrow(...). getReferenceById(...) jest przydatne, gdy wystarczy referencja po ID.

> Właściciel relacji
> W JPA zwykle strona z @ManyToOne i @JoinColumn przechowuje klucz obcy. Przykładowo OrderRequestEntity.client przechowuje client_id. Przy zapisie ustawiasz tę stronę; kolekcję ClientEntity.orderRequests zazwyczaj ignorujesz w domyślnym mapperze.

8. @Context - kiedy używać, a kiedy nie

@Context przekazuje podczas mapowania obiekt pamięci podręcznej. Dzięki IdentityHashMap można zwrócić wcześniej zmapowaną instancję zamiast mapować ją kolejny raz.

```
public class CycleAvoidingMappingContext {
private final Map<Object, Object> knownInstances = new IdentityHashMap<>();

@BeforeMapping
public <T> T getMappedInstance(Object source, @TargetType Class<T> targetType) {
return (T) knownInstances.get(source);
}

@BeforeMapping
public void storeMappedInstance(Object source, @MappingTarget Object target) {
knownInstances.put(source, target);
}
}
```

```
Client toDomain(
ClientEntity entity,
@Context CycleAvoidingMappingContext context
);
```

Wszystkie zagnieżdżone metody, które mają korzystać z tego zabezpieczenia, muszą przyjmować ten sam parametr @Context. Przy każdym najwyższym wywołaniu tworzysz nowy context.

> Ograniczenie
> @Context zatrzymuje rekurencję MapStruct, ale wynik nadal może być prawdziwie cykliczny: Client → OrderRequest → ten sam Client. Jackson przy serializacji JSON może wtedy nadal mieć problem.

> Ograniczenie w Twoim Domain
> Twoje klasy domain są budowane przez Lombok @Builder i nie mają setterów. Standardowy context najłatwiej działa z mutowalnymi obiektami, które można zapamiętać przed zmapowaniem pól zagnieżdżonych. Z builderami wymaga to dodatkowego projektu lub ręcznego składania grafu.

Wniosek: @Context jest narzędziem dla pełnych grafów używanych wewnętrznie. Dla REST API nadal wybieraj DTO bez cykli.

9. Praktyczny plan dla ZP Tank

Nie musisz przebudować wszystkiego jednego dnia. Najpierw napraw ścieżki, które wywołują błąd, a potem konsekwentnie wprowadzaj wzorzec.

Krok	Co zrobić	Dlaczego
1	Dodaj ClientSummaryDTO, OrderRequestSummaryDTO, OrderSummaryDTO.	Bezpieczne osadzanie danych w innych odpowiedziach.
2	Zmień OrderRequestDTO.client z ClientDTO na ClientSummaryDTO.	Koniec cyklu z Client.orderRequests.
3	Zmień OrderRequestDTO.order na OrderSummaryDTO.	Koniec cyklu Order.orderRequests.
4	Usuń pełnego ClientDTO z DeliveryAddressDTO lub zastąp clientId/ClientSummaryDTO.	Koniec cyklu adres - klient.
5	Dodaj @Named i uses dla zagnieżdżonych mapperów.	Wybierasz świadomie wariant mapowania.
6	W Entity mapperach dodaj warianty withoutRelations / withoutClient.	Zabezpieczasz Entity ↔ Domain.
7	Dodaj Create/Update DTO z clientId, productId itd.	Relacje ustawiasz w serwisie, nie przez pełny graf.
8	Dodaj endpoint /clients/{id}/order-requests.	Lista zleceń bez rozwijania całego klienta.

Checklist przed dodaniem nowej relacji

• Czy DTO A zawiera DTO B? • Czy DTO B może z powrotem zawierać pełne DTO A? • Czy do odpowiedzi wystarczy SummaryDTO? • Czy do zapisu wystarczy ID? • Czy mapper Entity ↔ Domain ma wariant bez relacji zwrotnej?

> Ostatnia zasada
> Projektuj odpowiedź HTTP od potrzeb ekranu, a nie od struktury tabel. Encje mogą być bogate i połączone; DTO powinny być celowe, małe oraz wolne od przypadkowych cykli.